



Matplotlib

- What is it?
- Why Matplotlib?
- Components fundamentals
- Mode
- Styles
- Annotations
- Bar charts
- Scatterplots
- Time
- Cheatsheet

Matplotlib: What is it?

- **Matplotlib** is a *Python* library for *data visualization* .
- It features advanced features for creating charts, including animated and interactive ones
- It offers the possibility to customize graphs and export them in various formats.
- It is *open-source* and **freely accessible** (BSD, free license).

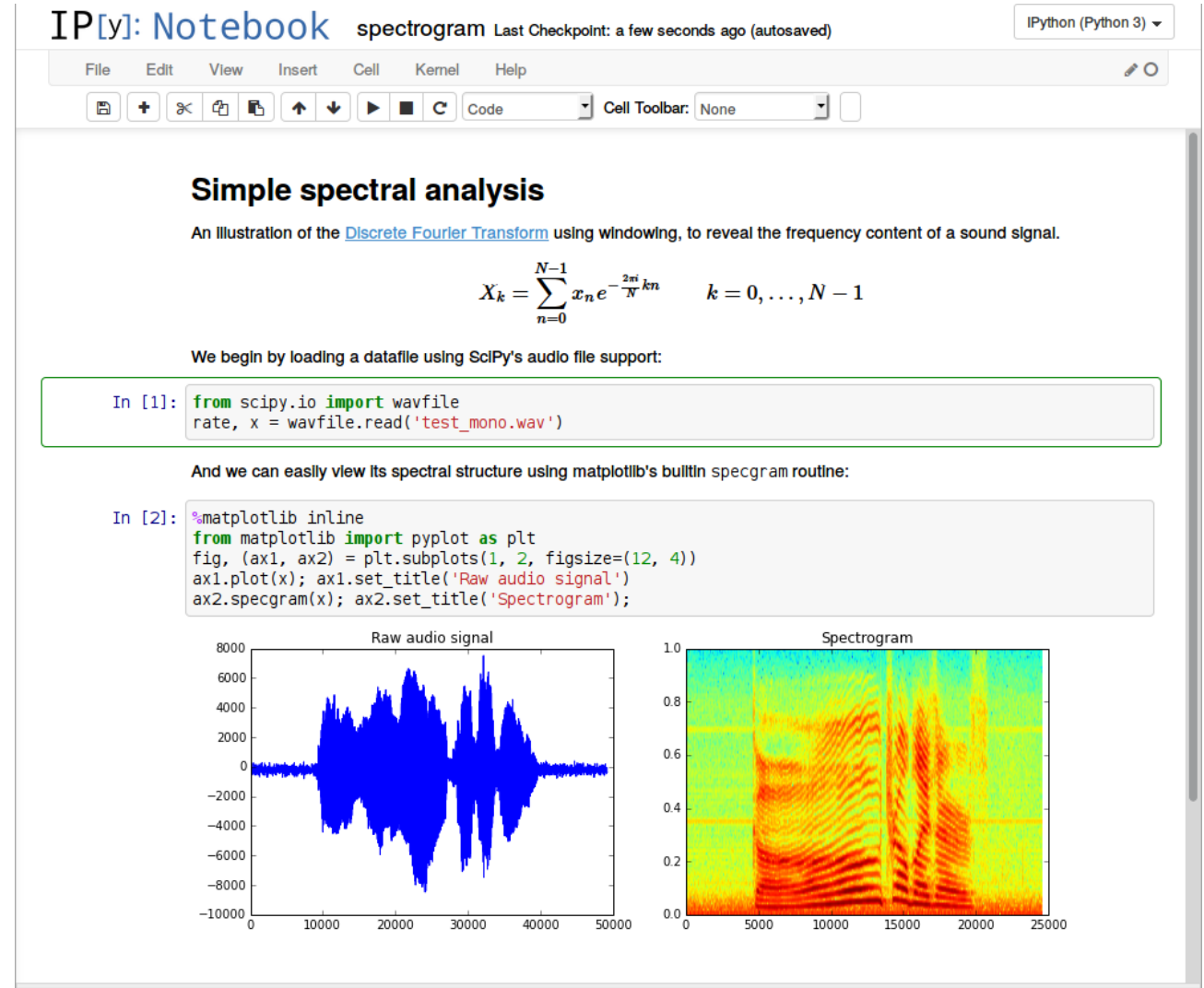


More docs at:

<https://matplotlib.org/stable/>

Matplotlib: Why?

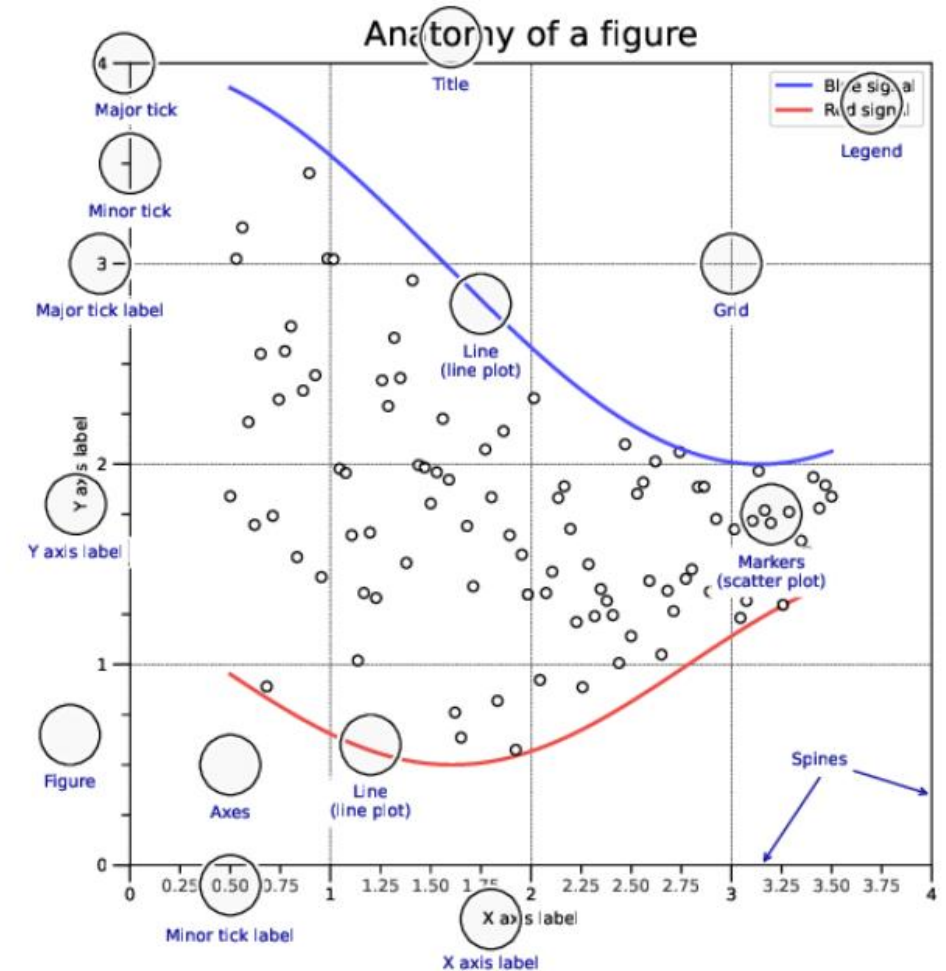
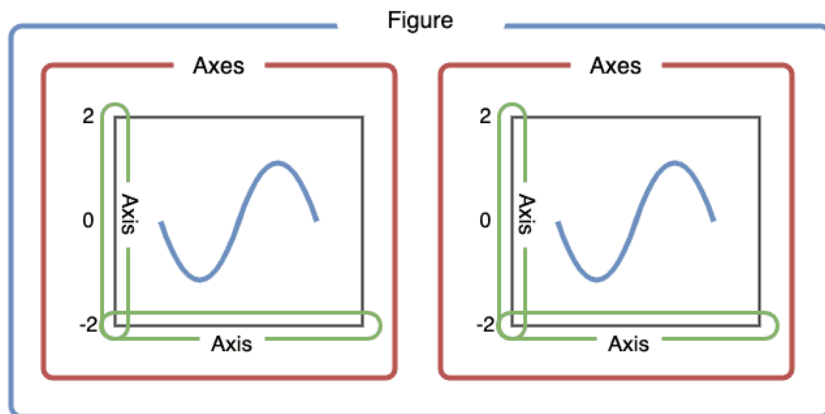
- Used to create quality graphics for reports and publications.
- Interactivity and customization are extensive.
- Integrated with different graphical interfaces or work environments, such as *Jupyter Notebooks*.
- Other widely used graphics libraries, such as **Seaborn**, are based on **Matplotlib**.



Matplotlib: Fundamental Components

The fundamental components of a figure are:

- **Figures** : container of all graphic elements
- **Axes** : area where data can be represented;
 - Each figure can contain one or more axes;
 - Each Axes can have one or more subplots (graphs).
- **Axis** : Axes representing scales and intervals
- **Labels** : labels, titles, etc...
- **Legend** : legend.



Matplotlib: Modes



In Matplotlib there are 2 fundamental ways to build graphs, i.e., 2 fundamental ways to manage the functions that allow us to build or modify graphs as we like:

1. *Object-Oriented mode*
2. *pyplot interface mode*

Matplotlib: Usage Modes

Object-Oriented Mode

```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    fig, ax = plt.subplots()
    ax.plot(x, x, label='linear')
    ax.plot(x, x**2, label='quadratic')
    ax.plot(x, x**3, label='cubic')
    ax.set_xlabel('x label')
    ax.set_ylabel('y label')
    ax.set_title("Simple Plot")
    ax.legend()
    plt.show()
```

Pyplot interface Mode

```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    plt.figure()
    plt.plot(x, x, label='linear')
    plt.plot(x, x**2, label='quadratic')
    plt.plot(x, x**3, label='cubic')
    plt.xlabel('x label')
    plt.ylabel('y label')
    plt.title("Simple Plot")
    plt.legend()
    plt.show()
```

Matplotlib: Mode

Object-Oriented Mode

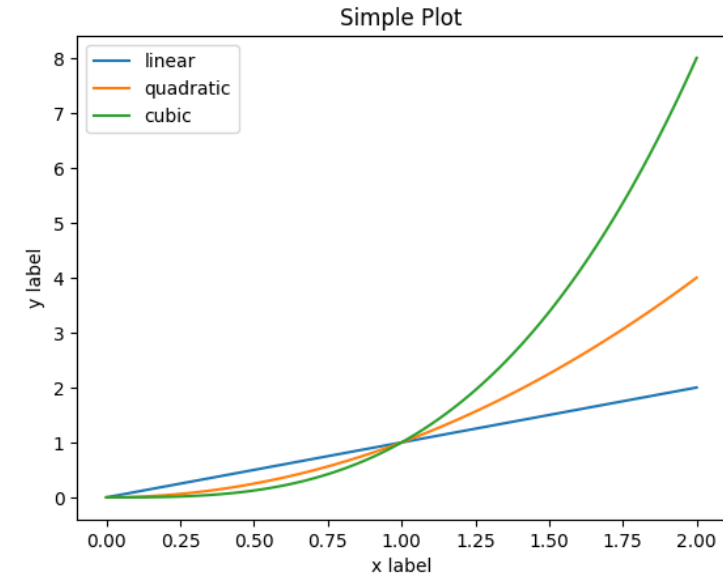
```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    fig, ax = plt.subplots()
    ax.plot(x, x, label='linear')
    ax.plot(x, x**2,
            label='quadratic')
    ax.plot(x, x**3,
            label='cubic')
    ax.set_xlabel('x label')
    ax.set_ylabel('y label')
    ax.set_title("Simple Plot" )
    ax.legend()
    plt.show()
```

Pyplot interface Mode

```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    plt.figure()
    plt.plot(x, x,
             label='linear')
    plt.plot(x, x**2,
             label='quadratic')
    plt.plot(x, x**3,
             label='cubic' )
    plt.xlabel('x label')
    plt.ylabel('y label')
    plt.title("Simple Plot")
    plt.legend()
    plt.show()
```

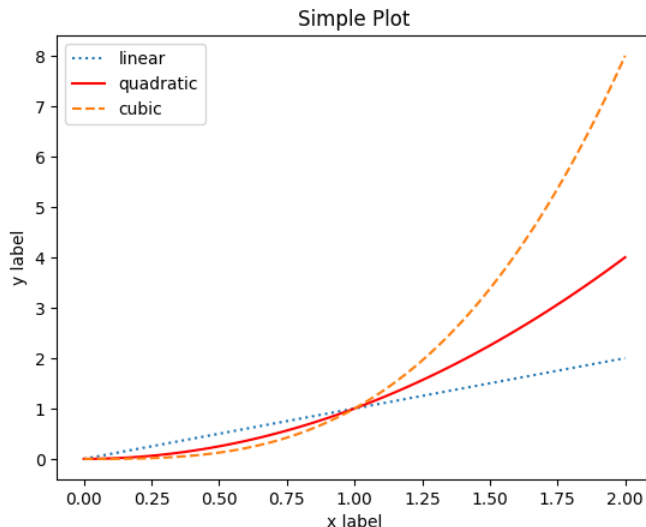


Matplotlib: Styles

- Let's add the colors of the graph lines,
- We increase the thickness to "2",
- Let's change the style of the lines.
- *Run with pyplot interface*

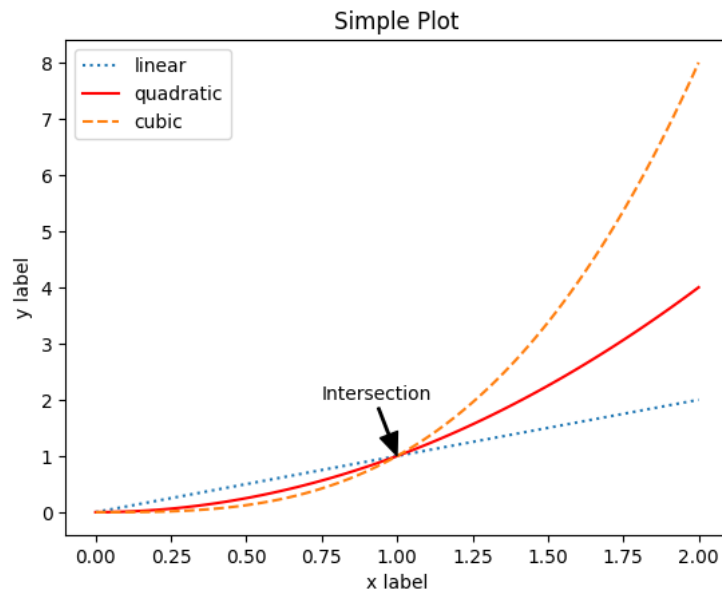
```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 10)
    plt.figure()
    plt.plot(x, x, 'd', label='linear', color='blue',
             linewidth=2, linestyle='--')
    plt.plot(x, x**2, 's', label='quadratic',
             color='red', linewidth=2, linestyle=None)
    plt.plot(x, x**3, label='cubic', color='orange',
             linewidth=2, linestyle=':')
    plt.xlabel('x label')
    plt.ylabel('y label')
    plt.title("Simple Plot")
    plt.legend()
    plt.show()
```



Matplotlib: Annotations

- Let's add an annotation in the graph
- Executed in OOP mode*

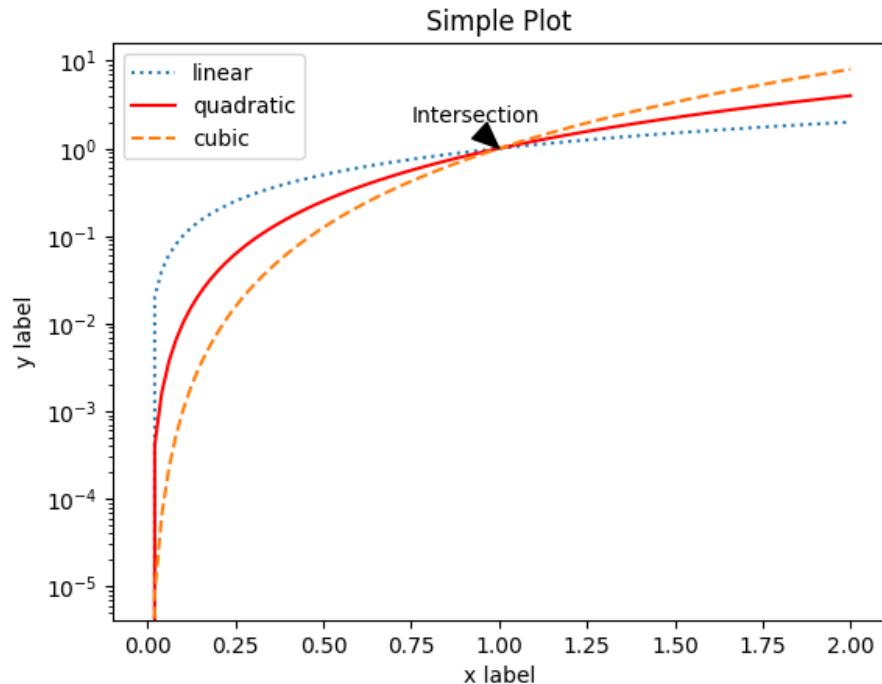


```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    fig, ax = plt.subplots()
    ax.plot(x, x, 'd', label='linear', color='blue',
            linewidth=2, linestyle='--')
    ax.plot(x, x**2, 's', label='quadratic',
            color='red', linewidth=2, linestyle=None)
    ax.plot(x, x**3, label='cubic', color=orange,
            linewidth=2, linestyle=':')
    ax.annotate('Intersection', xy=(1, 1), xytext=(0.75,
2),
                arrowprops=dict(facecolor='black', width=1,
                                headwidth=10, shrink=0.01))
    ax.set_xlabel('x label')
    ax.set_ylabel('y label')
    ax.set_title("Simple Plot")
    ax.legend()
    plt.show()
```

Matplotlib: Annotations

- Let's set the y-axis to a logarithmic scale
- *Executed in OOP mode*

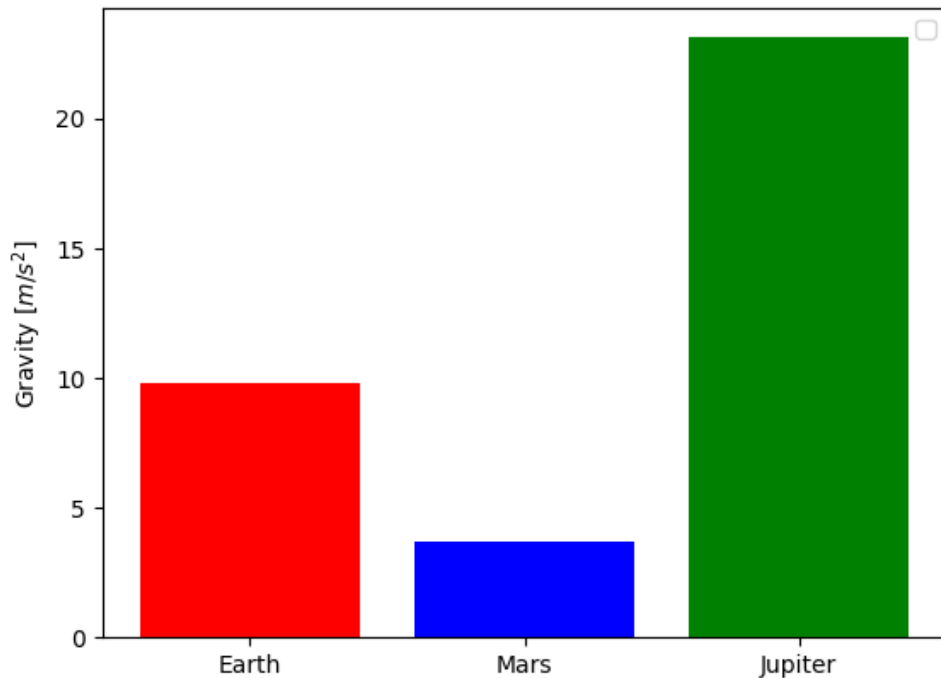


```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2, 100)
    fig, ax = plt.subplots()
    ax.plot(x, x, 'd', label='linear', color='blue',
            linewidth=2, linestyle='--')
    ax.plot(x, x**2, 's', label='quadratic',
            color='red', linewidth=2, linestyle=None)
    ax.plot(x, x**3, label='cubic', color='orange',
            linewidth=2, linestyle=':')
    ax.annotate('Intersection', xy=(1, 1),
                xytext=(0.75, 2), arrowprops=
                dict(facecolor='black', width=1, headwidth=10,
                    shrink=0.01))
    ax.set_yscale('log')
    ax.set_xlabel('x label')
    ax.set_ylabel('y label')
    ax.set_title("Simple Plot")
    ax.legend()
    plt.show()
```

Matplotlib: Bar Charts

- Very useful for comparing numerical quantities
- Associate a numerical value with each category (or class)
- *How are graphic objects declared?*

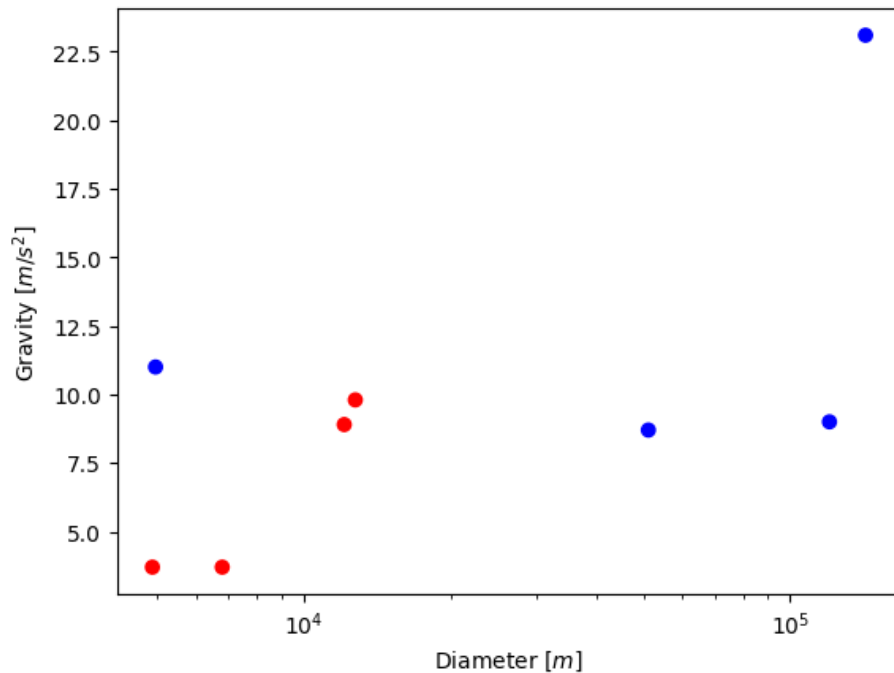


```
import matplotlib.pyplot as plt
import pandas as pd
```

```
if __name__ == '__main__':
    df = pd.read_csv("planets.csv")
    fig, ax = plt.subplots()
    ax.bar(df[ "Planet" ], df[
        "Gravity" ], color=('red',
        'blue', 'green'))
    ax.legend()
    ax.set_ylabel("Gravity [$m/s^2$]")
    plt.show()
```

Matplotlib: Scatterplots

- Scatterplot: graph of points
- Useful for checking how data is distributed in space
- *How are graphic objects declared?*

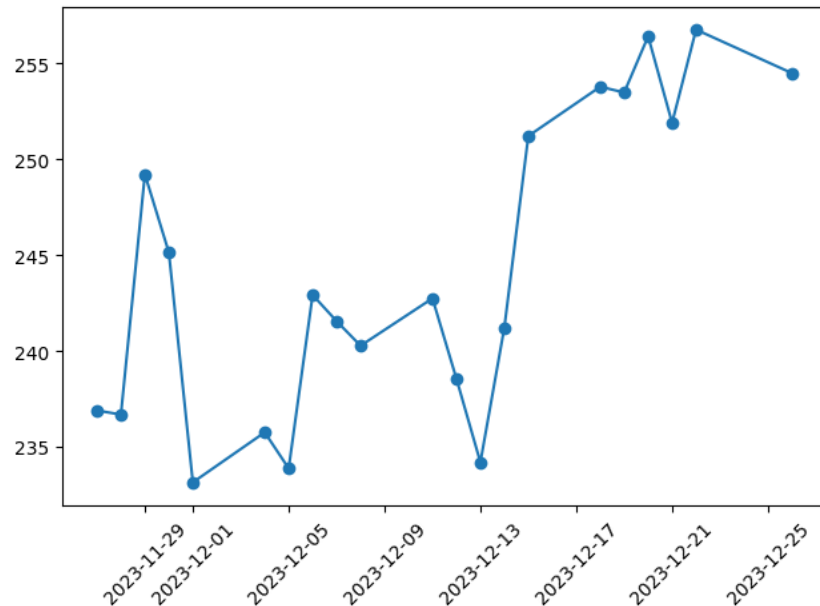


```
import matplotlib.pyplot as plt
import pandas as pd

if __name__ == '__main__':
    df = pd.read_csv("planets.csv")
    df['Type'] = ['Rocky', 'Rocky',
                  'Rocky',          'Rocky', 'Gas', 'Gas',
                  'Gas', 'Gas']
    fig, ax = plt.subplots()
    colors = {'Rocky': 'red', 'Gas': 'blue'}
    ax.scatter(df["Diameter"],
               df["Gravity"], c
               =df['Type'].map(colors))
    ax.set_xscale('log')
    ax.set_xlabel("Diameter [$m$]")
    ax.set_ylabel("Gravity [$m/s^2$]")
    plt.show()
```

Matplotlib:Time Series

- They are usually represented with line or point graphs
- On the x axis there is the time, on the y axis the measured quantity
- You need to pay attention in the ***preprocessing phase*** to how time is encoded on the x-axis.
- Pandas offers several features to manage temporal information.
- *How are graphic objects declared?*



```
import pandas as pd
import matplotlib.pyplot as plt

if __name__ == '__main__':
    df = pd.read_csv("HistoricalData_TSLA.csv")
    timesteps = df["Date"].astype("datetime64[s]")
    prices = df["Open"].str.replace('$',
                                     '').astype("float64")
    fig, ax = plt.subplots()
    ax.plot(timesteps, prices, marker='o',
            linestyle='--')
    ax.tick_params(axis='x', rotation=45)
    plt.tight_layout()
    plt.show()
```

Matplotlib – Cheat Sheet



Quick start

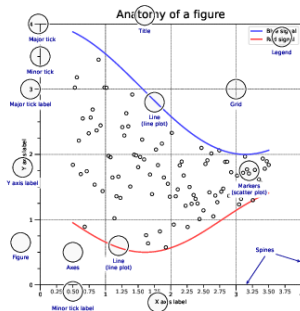
```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
X = np.linspace(0, 2*np.pi, 100)
Y = np.cos(X)
```

```
fig, ax = plt.subplots()
ax.plot(X, Y, color='green')
```

```
fig.savefig("figure.pdf")
plt.show()
```

Anatomy of a figure



Subplots layout

```
subplot[s](rows,cols,...)
fig, axs = plt.subplots(3, 3)
```

```
G = gridspec(rows,cols,...)
ax = G[0,:]
```

```
ax.inset_axes(extent)
```

```
d=make_axes_locatable(ax)
ax = d.new_horizontal('10%')
```

Getting help

• matplotlib.org
• github.com/matplotlib/matplotlib/issues
• discourse.matplotlib.org
• stackoverflow.com/questions/tagged/matplotlib
• https://gitter.im/matplotlib/matplotlib
• twitter.com/matplotlib
• Matplotlib users mailing list

Basic plots

```
plot([X],Y,[fmt],...)
X,Y,fmt,color,marker,linestyle
```

```
scatter(X,Y,...)
X,Y,[s]izes,[c]olors,marker,cmap
```

```
bar[h](x,height,...)
x,height,width,bottom,align,color
```

```
imshow(Z,...)
Z,cmap,interpolation,extent,origin
```

```
contour[f]([X],[Y],Z,...)
X,Y,Z,levels,color,extent,origin
```

```
pcolormesh([X],[Y],Z,...)
X,Y,Z,vmin,vmax,cmap
```

```
quiver([X],[Y],U,V,...)
X,Y,U,V,C,units,angles
```

```
pie(X,...)
Z,explode,labels,color,radius
```

```
text(x,y,text,...)
x,y,text,va,ha,size,weight,transform
```

```
fill[_between][x](...)
X,Y1,Y2,color,where
```

Advanced plots

```
step(X,Y,[fmt],...)
X,Y,fmt,color,marker,where
```

```
boxplot(X,...)
X,notch,sym,bootstrap,widths
```

```
errorbar(X,Y,xerr,yerr,...)
X,Y,xerr,yerr,fmt
```

```
hist(X,bins,...)
X,bins,range,density,weights
```

```
violinplot(D,...)
D,positions,widths,vert
```

```
barbs([X],[Y],U,V,...)
X,Y,U,V,C,length,pivot,sizes
```

```
eventplot(positions,...)
positions,orientation,lineoffsets
```

```
hexbin(X,Y,C,...)
X,Y,C,gridsize,bins
```

Scales

```
ax.set_[xy]scale(scale,...)
linear any values
symlog any values
log log values > 0
logit 0 < values < 1
```

```
Projections
subplot(...,projection=p)
p='polar'
p='3d'
```

```
p=ccrs.Orthographic()
import cartopy.crs as ccrs
```

```
Lines
linestyle or ls
capstyle or dash_capstyle
"butt" "round" "projecting"
```

```
Markers
```

```
Colors
```

```
Colormaps
```

```
Uniform
```

```
Sequential
```

```
Diverging
```

```
Qualitative
```

```
Cyclic
```

Tick locators

```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_locator(locator)
```

```
ticker.NullLocator()
```

```
ticker.MultipleLocator(5,5)
```

```
ticker.FixedLocator([0, 1, 5])
```

```
ticker.LinearLocator(numticks=3)
```

```
ticker.IndexLocator(base=0.5,offset=0.25)
```

```
ticker.AutoLocator()
```

```
ticker.MaxNLocator(n=4)
```

```
ticker.LogLocator(base=10, numticks=15)
```

```
Tick formatters
```

```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_formatter(formatter)
```

```
ticker.NullFormatter()
```

```
ticker.FixedFormatter(['zero', 'one', 'two', ...])
```

```
ticker.FuncFormatter(lambda x, pos: "%2.2f" % x)
```

```
ticker.FormatStrFormatter('%d')
```

```
ticker.ScalarFormatter()
```

```
ticker.StrMethodFormatter('{x}')
```

```
ticker.PercentFormatter(xmax=5)
```

```
Ornaments
```

```
ax.legend(...)
handles, labels, loc, title, frameon
```

```
ax.colorbar(...)
mappable, ax, cax, orientation
```

```
ax.annotate(...)
text, xy, xytext, sycords, textcoords, arrowprops
```

```
Event handling
```

```
fig, ax = plt.subplots()
def on_click(event):
    print(event)
fig.canvas.mpl_connect(
    'button_press_event', on_click)
```

Animation

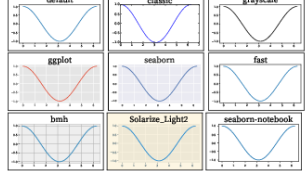
```
import matplotlib.animation as mpla
```

```
T = np.linspace(0, 2*np.pi, 100)
S = np.sin(T)
```

```
line, = plt.plot(T, S)
def animate(i):
    line.set_ydata(np.sin(T+i/50))
anim = mpla.FuncAnimation(
    plt.gcf(), animate, interval=5)
plt.show()
```

Styles

```
plt.style.use(style)
```



Quick reminder

```
ax.grid()
ax.set_[xy]lim(vmin, vmax)
ax.set_[xy]label(label)
ax.set_[xy]ticks(ticks, [labels])
ax.set_[xy]ticklabels(labels)
ax.set_title(title)
ax.tick_params(width=10, ...)
ax.set_axis_[on|off]()
```

```
fig.suptitle(title)
fig.tight_layout()
plt.gcf(), plt.gca()
mpl.rc('axes', linewidth=1, ...)
[fig|ax].patch.set_alpha(0)
text=r'$\frac{e^{-i\pi}}{2}$'
```

Keyboard shortcuts

Ctrl+S	Save	Ctrl+W	Close plot
F	Reset view	F	Fullscreen 0/1
F	View forward	B	View back
P	Pan view	O	Zoom to rect
X	X pan/zoom	Y	Y pan/zoom
G	Minor grid 0/1	G	Major grid 0/1
L	X axis log/linear	L	Y axis log/linear

Ten simple rules

1. Know your audience
2. Identify your message
3. Adapt the figure
4. Captions are not optional
5. Do not trust the defaults
6. Use color effectively
7. Do not mislead the reader
8. Avoid "chartjunk"
9. Message trumps beauty
10. Get the right tool

Links